

NUMPY · PANDAS · CSV

SQLALCHEMY · DUCKDB · CSV

1. Tipos de variables ↔ Check de variables de una tabla

```
# tipo nativo / dtype
type(x)
x.dtype
df.dtypes
df.dtypes.value_counts()      # conteo por tipo
np.array([1, 2]).dtype
pd.Series([1.0, 2.0]).dtype
df.select_dtypes(include="number") # filtrar por tipo

# SQLAlchemy
from sqlalchemy import inspect
insp = inspect(engine)
insp.get_columns("tabla")
[c["name"] for c in insp.get_columns("tabla")]

-- DuckDB
DESCRIBE tabla;
PRAGMA table_info('tabla');
DESCRIBE SELECT * FROM 'archivo.csv';
SELECT typeof(a) FROM tabla LIMIT 1;
```

2. Declaración de variables ↔ Cambio de tipo de variable

```
x = np.array([1, 2, 3], dtype=np.int32)
s = pd.Series([1, 2, 3])
df = pd.DataFrame({"a": [1, 2], "b": [3, 4]})
z = np.zeros((3, 3)); o = np.ones(5); r = np.arange(0, 10, 2)

x = x.astype(np.float64)
df["a"] = df["a"].astype("category")
df["b"] = pd.to_numeric(df["b"], errors="coerce")
df["fecha"] = pd.to_datetime(df["fecha"])

# declaración (SQLAlchemy Core)
from sqlalchemy import Column, Integer, String, Table, MetaData
tabla = Table("t", MetaData(),
              Column("a", Integer), Column("b", String))

-- cambio de tipo
ALTER TABLE tabla ALTER COLUMN a TYPE DOUBLE;
SELECT CAST(a AS DOUBLE) AS a_float FROM tabla;
SELECT a::DOUBLE AS a_float FROM tabla; -- shorthand DuckDB
```

3. Inspección de dimensiones ↔ Inspección a fondo de atributos

```
x.shape, x.ndim, x.size
df.shape
df.columns.tolist()
df.index
df.info()
df.memory_usage(deep=True).sum()
df.describe()      # estadísticas numéricas
df.nunique()        # valores únicos por columna

SELECT COUNT(*) FROM tabla;      -- nº filas
PRAGMA table_info('tabla');      -- columnas+tipo+nulos
SUMMARIZE tabla;                 -- estadísticas (DuckDB)
SELECT COUNT(DISTINCT a) FROM tabla; -- valores únicos

insp.get_columns("tabla")
insp.get_table_names()           # listar tablas
session.query(Table).count()
```

4. Operaciones vectoriales simples (índice/columnas) ↔ Easy filtering

```
x[0], x[:, 1], x[1:3]      # indexado numpy
x[x > 5]                  # filtrado booleano numpy
df.iloc[0]               # por posición
df.loc[:, "a"]           # por etiqueta
df.loc[df["a"] > 5, ["a","b"]] # filtro + selección cols
df[df["a"] > 5]
df.drop(columns=["b"])
df.drop(index=[0, 1])
```

```
SELECT * FROM tabla WHERE a > 5;
SELECT a, c FROM tabla;           -- descartar cols
SELECT * FROM tabla LIMIT 10 OFFSET 20;
DELETE FROM tabla WHERE a IS NULL;
```

```
-- DuckDB directo sobre el CSV, sin cargarlo entero
SELECT * FROM 'archivo.csv' WHERE a > 5;
```

5. Combinación de valores (numéricas/strings) ↔ Operaciones en lectura (eficiencia)

```
df["c"] = df["a"] * df["b"]
df["media"] = df[["a", "b"]].mean(axis=1)
df["mediana"] = df[["a", "b"]].median(axis=1)
df["nombre_completo"] = df["nombre"] + " " + df["apellido"]
df["cat"] = df["cat"].str.upper().str.strip()
df["cat"] = df["cat"].str.replace(", ", "", regex=False)
df.groupby("cat")["a"].agg(["mean", "sum", "count"])
```

```
-- cálculo dentro del motor, no se materializa en Python
SELECT a * b AS c, AVG(a) OVER () AS media_a,
       MEDIAN(a) OVER () AS mediana_a
FROM tabla;
SELECT cat, AVG(a), SUM(a), COUNT(*) FROM tabla GROUP BY cat;
```

```
-- DuckDB: cálculo+filtro sobre CSV sin cargarlo en memoria
SELECT a * b AS c FROM read_csv_auto('archivo.csv')
WHERE a IS NOT NULL;
```

6. Carga de datos (CSV/SQL) ↔ manejo de índice

```
df = pd.read_csv("archivo.csv")
df = pd.read_csv("archivo.csv", index_col=0)
df = pd.read_csv("archivo.csv", usecols=["a","b"]) # solo cols
df = pd.read_csv("archivo.csv", nrows=1000)       # muestreo
df = df.reset_index(drop=True)
df = df.set_index("id")
```

```
-- DuckDB: tabla directa desde CSV
CREATE TABLE t AS SELECT * FROM read_csv_auto('archivo.csv');
CREATE TABLE t AS SELECT * FROM read_csv_auto('archivo.csv', header=True);
```

```
df = pd.read_sql_table("tabla", engine, index_col="id")
df = pd.read_sql("SELECT * FROM tabla", engine)
df = pd.read_sql("SELECT * FROM tabla", engine, index_col="id")
```

7. Descarga de datos (CSV/SQL) ↔ manejo de índice

```
df.to_csv("salida.csv", index=False)
df.to_csv("salida.csv", index=True)
df.to_csv("salida.csv", columns=["a","b"], sep=";")
df.to_parquet("salida.parquet", index=False) # más eficiente

-- DuckDB
COPY tabla TO 'salida.csv' (HEADER, DELIMITER ',');
COPY (SELECT * FROM tabla WHERE a > 5) TO 'filtrado.csv';

df.to_sql("tabla", engine, if_exists="replace", index=False)
df.to_sql("tabla", engine, if_exists="append", index=False)
```

8. Data cleaning: NaN, columnas, inplace ↔ en SQL y CSV

```
df.dropna(inplace=True)
df.dropna(subset=["a","b"], inplace=True) # solo esas cols
df.fillna(0, inplace=True)
df.fillna({"a": 0, "b": "N/A"}, inplace=True) # por columna
df.drop(columns=["col_x"], inplace=True)
df.drop_duplicates(subset=["id"], inplace=True)

DELETE FROM tabla WHERE a IS NULL;
UPDATE tabla SET a = 0 WHERE a IS NULL;
ALTER TABLE tabla DROP COLUMN col_x;
DELETE FROM tabla WHERE rowid NOT IN
  (SELECT MIN(rowid) FROM tabla GROUP BY id); -- dedupe

-- DuckDB directo sobre el CSV, sin pasar por pandas
SELECT DISTINCT * FROM read_csv_auto('archivo.csv') WHERE a IS NOT NULL;
```

9. Síntesis vectorizada: máscaras booleanas + lambda/apply ↔ SQL y CSV

```
mask = (df["a"] > 0) & (df["b"].notna())
df.loc[mask, "flag"] = 1
df["c"] = df["a"].apply(lambda x: x*2 if x > 0 else 0)
df["d"] = np.where(df["a"] > 0, "pos", "neg") # preferible a apply
df["e"] = np.select(
    [df["a"] > 10, df["a"] > 0], ["alto", "bajo"], default="cero")

SELECT *,
  CASE WHEN a > 0 THEN 1 ELSE 0 END AS flag,
  CASE WHEN a > 0 THEN a * a ELSE 0 END AS c,
  CASE WHEN a > 10 THEN 'alto'
    WHEN a > 0 THEN 'bajo' ELSE 'cero' END AS e
FROM tabla
WHERE a > 0 AND b IS NOT NULL;
```